

std::move と std::forward

ムーブセマンティクス 完全再入門

— C++11 → C++23 まで —

ゲーム開発のための C++ 講義

この講義のゴール

コピーで成り立っていた古い C/C++ から、

「無駄なメモリコピーを消す」ために追加された仕組み一式を理解する。

- **ゼロオーバーヘッド**：使わない機能にコストを払わない C++ の哲学
- ゲーム開発では、巨大なリソース（画像・メッシュ・音声）や、
毎フレーム触るコンテナが山ほどある
- だからこそ「コピーせずに所有権だけ渡す＝ムーブ」が効いてくる

⚠ 最初に一番大事なこと：**ムーブ** ≠ `std::move`

`std::move` は「ムーブして良いよ」という目印を付けるだけで、実際のコピー削除（ムーブ動作）を行うのはコンストラクタや代入演算子。

目次

1. 値カテゴリ (左辺値 / 右辺値)
2. コピーの復習 (浅い / 深い)
3. ムーブコンストラクタ・ムーブ代入演算子
4. `std::move` の正体
5. Rule of Five / Rule of Zero
6. `noexcept` とムーブ (なぜ重要か)
7. テンプレートと転送参照・`auto&&`
8. `std::forward` — 完全転送
9. **C++23 : deducing this** と `std::forward_like` **C++23**
10. スマートポインタ・`std::move_only_function`
11. `emplace_back` とコンテナ
12. コピー省略 (RVO / NRVO / 保証されたコピー省略)
13. まとめ・チートシート・落とし穴

1. 値カテゴリ

左辺値 (lvalue) と 右辺値 (rvalue)

「値」と言いつつ、実は「式」の分類

`int a = 10;` の `a` や `10` などの**式**を、**値カテゴリ**で分類する。

- **左辺値 (lvalue)**：名前があり、アドレスが取れるもの。次の行でも使える。

例：変数 `a`、配列要素 `v[0]`、`*ptr`

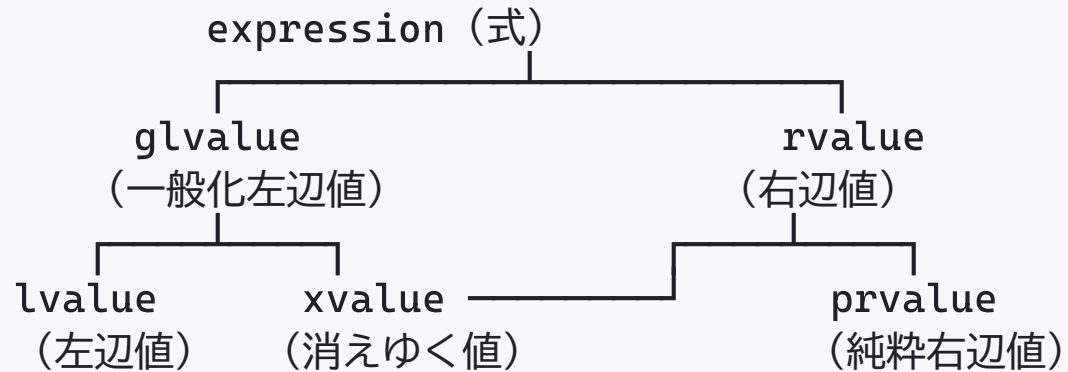
- **右辺値 (rvalue)**：一時的で、その場限りのもの。名前がない。

例：リテラル `10`、計算結果 `a + b`、関数が返す一時オブジェクト

```
int a = 10;      // a は左辺値 / 10 は右辺値
int b = a + 5;   // a + 5 は右辺値 (一時的な結果)
&a;             // OK: 左辺値はアドレスが取れる
// &(a + 5);    // NG: 右辺値のアドレスは取れない
```

C++11 以降は 5 分類に細分化された

「破棄していい一時オブジェクト (xvalue)」を表現するために分類が増えた。



- **prvalue** : 純粋な一時オブジェクト (`T{}`, `a+b`, 関数の戻り値の一時)
- **xvalue** (eXpiring value) : **もう使い終わって奪って良い**左辺値。
`std::move(x)` の結果がこれ
- **glvalue** : lvalue と xvalue の総称 / **rvalue** : xvalue と prvalue の総称

👉 実務の勘 : **名前があれば左辺値**、`std::move` を通すと **xvalue (= 右辺値扱い)**。

右辺値参照 `T&&` C++11

C++11 で追加された、**右辺値だけを束縛できる参照**。ムーブの土台。

```
int& lref = a;           // 左辺値参照：左辺値を束縛
int&& rref = 10;          // 右辺値参照：右辺値を束縛
int&& r2 = a + 5;         // OK：一時オブジェクトの寿命を延ばせる
// int&& r3 = a;          // NG：左辺値は束縛できない
```

- これで「**この式は一時オブジェクトだから中身を奪ってよい**」を型で区別できる
- 関数を `f(const T&)` と `f(T&&)` の 2 つに分ければ、
コピー版とムーブ版を**オーバーロードで振り分け**られる

2. コピーの復習

浅いコピー / 深いコピー

コピーコンストラクタ / コピー代入演算子

オブジェクトを**複製**する特殊メンバ関数。

```
class Buffer {  
    int*    data_;  
    size_t  size_;  
public:  
    Buffer(const Buffer& other)                // コピーコンストラクタ  
        : size_(other.size_), data_(new int[other.size_]) { // ★新しく確保  
        std::copy(other.data_, other.data_ + size_, data_); // ★中身も複製  
    }  
    Buffer& operator=(const Buffer& other);      // コピー代入演算子  
};
```

問題は「複製にコストがかかる」こと。

1MB のバッファをコピーすれば 1MB 分の確保 + 複写が毎回走る。

浅いコピー vs 深いコピー

浅い (shallow) コピー：ポインタだけ複製 → 実体は共有 (危険)



深い (deep) コピー：実体ごと複製 (安全だが重い)



- 生ポインタをメンバに持つクラスの**既定のコピー**は浅い → バグの温床
- 正しくは深いコピーを書く … が、**そもそも複製が不要なら？** → **ムーブ**の出番

3. ムーブ

ムーブコンストラクタ・ムーブ代入演算子

ムーブ = 「所有権の移動」

コピーは複製、ムーブは引越。中身を**新品確保せず、そのまま奪って**、元は空にする。

```
class Buffer {  
    int*    data_ = nullptr;  
    size_t size_ = 0;  
public:  
    Buffer(Buffer&& other) noexcept           // ★ムーブコンストラクタ  
        : data_(other.data_), size_(other.size_) { // ポインタを付け替えるだけ  
        other.data_ = nullptr;                // ★元は空にする (重要)  
        other.size_ = 0;  
    }  
};
```

ムーブ前	ムーブ後
src → [data]	src → nullptr
dst → null	dst → [data] ← 確保も複写もゼロ！

std::exchange で綺麗に書く C++14

「奪って、元を初期値に戻す」を 1 行で表現できる。

```
#include <utility>

Buffer(Buffer&& other) noexcept
: data_(std::exchange(other.data_, nullptr)), // other.data_ を奪い nullptr に
  size_(std::exchange(other.size_, 0)) {}

Buffer& operator=(Buffer&& other) noexcept {
    if (this != &other) {                    // 自己ムーブ対策
        delete[] data_;                      // 自分の資源を解放
        data_ = std::exchange(other.data_, nullptr);
        size_ = std::exchange(other.size_, 0);
    }
    return *this;
}
```

`std::exchange(obj, newval)` … obj に newval を代入し、**古い値を返す**。

ムーブ後のオブジェクトの状態

ムーブ元は消滅するわけではない。「有効だが未規定 (valid but unspecified)」な状態になる。

```
std::string s = "hello";  
std::string t = std::move(s); // s の中身は t へ  
  
// s はまだ存在する。破棄や再代入は安全。  
// ただし「中身が何か」は保証されない（空とは限らない）  
s = "world"; // ← 再代入は OK  
std::cout << s.size(); // ← 中身を前提にした利用は NG（未定義ではないが無意味）
```

⚠ **use-after-move（ムーブ後の中身の利用）** は典型的バグ。

ムーブした変数は「もう一度代入するまで中身を読まない」を鉄則に。

4. std::move の正体

std::move は「キャスト」でしかない

実行時に**何も動かさない**。ただ「これは右辺値 (xvalue) だ」と型を付け替えるだけ。

```
// 概念的には、これだけ↓ (実行時コストゼロ)
template <class T>
constexpr std::remove_reference_t<T>&& move(T&& t) noexcept {
    return static_cast<std::remove_reference_t<T>&&>(t);
}
```

- std::move(x) の結果が**右辺値**になる → **ムーブ用オーバーロード**が選ばれる
- 実際のコピー削除を行うのは**ムーブコンストラクタ／代入演算子**の中身
- だから **ムーブ** ≠ std::move 。 std::move は「奪って良い」という許可証

const はムーブできない (罠)

```
void foo(const std::string p) {  
    std::string q = std::move(p); // ムーブしたつもり...だがコピーされる！  
}
```

- ムーブコンストラクタは中身を書き換えるため、引数は**非 const 参照 (T&&)**
- `const T` を `std::move` すると `const T&&` になり、**ムーブ版に束縛できない**
- 結果、`const T&` (コピー版) に静かにフォールバックする

教訓：**ムーブしたい値に安易に `const` を付けない。**

「なんとなく `const`」がムーブを殺す。

5. Rule of Five / Rule of Zero

5 つの特殊メンバ関数 C++11

リソースを直接管理するクラスは、この 5 つをセットで考える (**Rule of Five**)。

メンバ	役割
デストラクタ	資源の解放
コピーコンストラクタ	複製して生成
コピー代入演算子	複製して代入
ムーブコンストラクタ	奪って生成
ムーブ代入演算子	奪って代入

`= default` (コンパイラ生成) / `= delete` (禁止) で明示できる。

※ どれか 1 つでも自分で書くと、他の自動生成が抑制される場合がある。

いちばんの推奨：Rule of Zero

自分で特殊メンバ関数を書かないのが最良。

資源管理は `std::vector` や `std::unique_ptr` などの **RAIIな型に任せる**。

```
class Player {  
    std::string          name_;    // 各メンバが自分で  
    std::vector<int>      scores_; // コピー/ムーブを正しく処理してくれる  
    std::unique_ptr<Mesh> mesh_;   // ので...  
    // 特殊メンバ関数は 1 つも書かない!  
    // → コンパイラが正しいコピー/ムーブを自動生成  
};
```

- 生ポインタ・生の `new/delete` を **メンバに持たない**
- ムーブもコピーも「メンバ任せ」で正しく動く
- **C++20** 条件付きトリビアル特殊メンバにより、最適化もさらに素直に

6. noexcept とムーブ

なぜセットで語られるのか

noexcept は「例外を出さない」保証 C++11

```
Buffer(Buffer&& other) noexcept; // この関数は例外を投げないとコンパイラに約束
```

- コンパイラ・標準ライブラリが最適化の判断に使う
- 破ると（実際に例外が出ると） `std::terminate` でプログラム停止

ムーブでこれが決定的に効くのが `std::vector` の再確保。

std::vector は move_if_noexcept で判断する

vector が容量を増やすとき、既存要素を新領域へ移す。このとき：

ムーブコンストラクタが `noexcept` → ムーブで移す (速い) ✓
ムーブコンストラクタが `noexcept` でない → コピーで移す (遅い) ✗

理由：再確保の途中で例外が出ても**元の配列を壊さない**保証 (強い例外保証) のため。
`noexcept` でないムーブは「途中で失敗すると復元できない」ので、安全なコピーを選ぶ。

```
// noexcept を付け忘れると...要素追加のたびに全コピーが走り、  
// せっかくのムーブが効かない。→ ムーブは基本 noexcept を付ける！
```

7. テンプレートと転送参照

T&& / auto&&

この章のゴール

第 8 章の `std::forward` を理解するための**準備運動**。
やりたいことは、たった 1 つ。

**1 つの関数で、左辺値も右辺値も受け取り、
「どちらで来たか」を覚えておきたい。**

- 左辺値で来たら → 相手にも **コピー** させたい
- 右辺値で来たら → 相手にも **ムーブ** させたい

そのカギが、これから説明する **転送参照** という特別な `T&&`。
用語が続くので、ゆっくり進みます。

おさらい：T&& は「右辺値参照」だった

第 1 章では、T&& は**右辺値だけを受け取る参照**だと学んだ。

```
void f(Widget&& w); // Widget の右辺値（一時オブジェクト）専用
f(Widget{});      // OK：右辺値
Widget a;
// f(a);          // NG：左辺値は渡せない
```

- ここでの Widget&& は **型が最初から決まっている** → ただの右辺値参照
- 左辺値 a は受け取れない

ところが、T が「テンプレートで推論される型」だと、同じ T&& の意味がガラッと変わる。次のページへ。

テンプレートの **T&&** は「両方」受け取れる

```
template <class T>
void g(T&& x);      // T はコンパイラが推論する → これは「転送参照」

Widget a;
g(a);              // OK: 左辺値も
g(Widget{});       // OK: 右辺値も、どちらも渡せる！
```

用語：転送参照 (forwarding reference)

テンプレートで **T** が推論されるときの **T&&**。左辺値も右辺値も受け取れる、特別な参照。別名 **ユニバーサル参照**。

⚠ 見た目は同じ **T&&** でも意味が違う：

- `void f(Widget&& w)` … 型が固定 → **ただの右辺値参照** (右辺値専用)
- `template<class T> void g(T&& x)` … **T** を推論 → **転送参照** (両方OK)

「型推論」で何が起きているのか

`g(x)` を呼ぶと、コンパイラは渡された引数を見て `T` を決める。
このとき **左辺値か右辺値かで、決まる `T` が変わる。**

```
Widget a;  
g(a);           // ← 左辺値を渡した  
g(Widget{});    // ← 右辺値を渡した
```

渡したもの	推論される <code>T</code>	引数 <code>x</code> の型
左辺値 <code>a</code>	<code>Widget&</code> (&が付く)	<code>Widget&</code> (左辺値参照)
右辺値 <code>Widget{}</code>	<code>Widget</code> (そのまま)	<code>Widget&&</code> (右辺値参照)

ポイントは、**左辺値で来ると `T` に `&` が付く** こと。
コンパイラが「左辺値か右辺値か」を、こっそり `T` に記録している。

参照の参照？ — reference collapsing

前ページで、左辺値のとき `T = Widget&` になった。すると `T&&` は…

`T&&` に `T = Widget&` を当てはめると：
`Widget& &&` ← 参照の参照になってしまう！

`Widget& &&` なんて自分では書けない。でもテンプレート経由だと現れる。
これをコンパイラが**単純なルールでつぶす** (reference collapsing)。

<code>T&</code>	<code>&</code>	<code>→ T&</code>	(<code>&</code> が 1 つでもあれば → 左辺値参照)
<code>T&</code>	<code>&&</code>	<code>→ T&</code>	
<code>T&&</code>	<code>&</code>	<code>→ T&</code>	
<code>T&&</code>	<code>&&</code>	<code>→ T&&</code>	(<code>&&</code> と <code>&&</code> のときだけ → 右辺値参照)

- 左辺値： `T=Widget&` → `Widget& &&` → **`Widget&`** (左辺値のまま)
- 右辺値： `T=Widget` → `Widget&&` → **`Widget&&`** (右辺値のまま)

👉 **結果、来たときの値カテゴリがそのまま保たれる。**

まとめ：転送参照は「どっちで来たか」を覚える箱

難しく見えるが、言いたいことは1つ。

転送参照 `T&&` は、**左辺値で来たか・右辺値で来たかを `T` に記録して保持する。**

左辺値で呼ぶ	→	<code>T = Widget&</code>	→	<code>x</code> は左辺値参照
右辺値で呼ぶ	→	<code>T = Widget</code>	→	<code>x</code> は右辺値参照

- これで「どちらで来たか」は保持できた
- でも…この `x` を**次の関数へ渡すと、情報が消えてしまう** (！)
- その落とし穴と解決策が、次章の `std::forward`

auto&& : 転送参照の「お手軽版」 C++11

auto&& も転送参照。仕組みは同じで、書き方が簡単なだけ。

```
for (auto&& e : container) { // ← 中身が何であれ、コピーせず受けられる
    process(e);
}
```

範囲 for で「要素を参照で受けたいが、型を書くのが面倒」なときに最適。

受け方	性質
const auto&	読み取り専用・非コピー（変更しないなら第一候補）
auto（値）	コピーが発生 （意図的でなければ避ける）
auto&&	何でも非コピーで受けられる汎用形。ジェネリックで特に有効

8. std::forward

完全転送 (perfect forwarding)

転送参照だけでは、情報が消える

第7章で「転送参照は、左辺値/右辺値を覚える」と学んだ。
ところが、その引数を**次の関数へ渡そうとすると問題が起きる**。

```
template <class T>
void wrapper(T&& x) { // x は転送参照 (左辺値/右辺値を覚えている)
    target(x);        // これを別の関数へ渡したい...
}
```

一見うまくいきそう。でも実は、**右辺値で来ても左辺値として渡ってしまう**。
なぜか？ 次のページで、その意外な理由を見ていく。

名前が付くと、右辺値も「左辺値」になる

C++ の超重要ルール：**名前の付いた変数は、式としては左辺値。**
たとえば「右辺値参照」として受け取っても、だ。

```
template <class T>
void wrapper(T&& x) {
    &x; // ← アドレスが取れる = x は左辺値！
}
```

- **x** は「右辺値を受け取る箱」だが、**箱そのものには名前がある**
- 名前があってアドレスが取れる → その式は左辺値

💡 イメージ：右辺値参照 **x** は「一時オブジェクトを指す名札」。
名札には名前があるので、名札自体を指すと左辺値になる。

だから、そのまま渡すとコピーになる

左辺値になった `x` を渡すと、**いつもコピー版が選ばれてしまう**。

```
void target(const Widget&); // コピー版
void target(Widget&&);      // ムーブ版

template <class T>
void wrapper(T&& x) {
    target(x);              // x は左辺値 → 常に「コピー版」が呼ばれる
}

wrapper(Widget{});         // 右辺値を渡したのに... → コピーされる！🤖
```

- せっかく右辺値（ムーブ可能）で渡したのに、ムーブされずコピーに
- 「来たときの値カテゴリ」を、渡すときに**復元**したい → `std::forward`

std::forward = 覚えた値カテゴリを復元する

std::forward<T> は、第 7 章で T に記録された「左辺値/右辺値」を見て、**元の値カテゴリへキャストし直す**。

```
template <class T>
void wrapper(T&& x) {
    target(std::forward<T>(x)); // T の記憶どおりに復元して渡す
}

wrapper(a);           // 左辺値で来た → T=Widget& → 左辺値のまま → コピー版
wrapper(Widget{});    // 右辺値で来た → T=Widget → 右辺値に復元 → ムーブ版 ✓
```

用語：完全転送 (perfect forwarding)

受け取った引数の値カテゴリ (左辺値/右辺値) を**崩さず**、そのまま次の関数へ渡すこと。それを実現する道具が std::forward。

move と forward の違い

どちらも「値カテゴリを変えるキャスト」だが、性格が違う。

	やること	ひとことと言うと
<code>std::move(x)</code>	無条件 に右辺値化	「もう使わない、奪ってOK」と断言
<code>std::forward<T>(x)</code>	T に応じて キャスト	「来たとおりに渡す」

```
std::move(x)           // 常に右辺値（ムーブを促す）
std::forward<T>(x)     // T=U& なら左辺値 / T=U なら右辺値
```

👉 `forward` は転送参照とセット。 `T` という「記憶」があって初めて働く。

move と forward の使い分け

```
template <class T>
void store(T&& x) {
    member_ = std::forward<T>(x);    // 転送参照で受けた → forward
}

void setName(std::string name) {
    name_ = std::move(name);         // 値で受けた (もう使わない) → move
}
```

覚え方

- 転送参照 `T&&` (型推論あり) で受けた引数 → `std::forward<T>`
- 具体的な右辺値参照 / これ以上使わないローカル値 → `std::move`

実例：完全転送するファクトリ

```
template <class T, class ... Args>
std::unique_ptr<T> make(Args&& ... args) {
    return std::unique_ptr<T>(
        new T(std::forward<Args>(args) ... )); // 引数を「そのまま」コンストラクタへ
}
```

- 呼び出し側が左辺値ならコピー、右辺値ならムーブで、**意図どおり**に届く
- `...` により、複数の引数もまとめて完全転送できる
- これがまさに `std::make_unique` / `emplace_back` の内部でやっていること

9. C++23 の新機能 C++23

deducing this と std::forward_like

この章の位置づけ

第 7・8 章で学んだ **転送参照** と `std::forward`。

C++23 では、それを土台にした新機能で「同じことがもっと楽に」書ける。

この章で学ぶのは 2 つ。

- **deducing this** (明示的オブジェクトパラメータ)
- `std::forward_like`

名前は難しそうだが、やることは「これまでの続き」。順番に見ていく。

準備：メンバ関数にも「左辺値/右辺値」がある

引数に左辺値/右辺値があったように、**呼び出す“オブジェクト自身”**にも区別がある。

```
Widget w;           // w は左辺値
w.get();            // 左辺値のオブジェクトから呼ぶ
std::move(w).get(); // 右辺値のオブジェクトから呼ぶ
Widget{}.get();      // 一時オブジェクト(右辺値)から呼ぶ
```

メンバ関数の後ろに `&` / `&&` を付けると、この違いで**呼び分け**できる。

```
struct Widget {
    void get() &; // 左辺値のオブジェクト用
    void get() &&; // 右辺値のオブジェクト用
};
```

用語：参照修飾子 (ref-qualifier) = メンバ関数の後ろに付ける `&` / `&&`。

呼び出し元オブジェクトが左辺値か右辺値かで、呼ぶ関数を分けられる。

なぜ 4 つも書くのか — getter の増殖

「オブジェクトの状態に合わせて、最適な返し方をしたい」と考えると…

```
class Widget {  
    std::string data_;  
public:  
    std::string&      get() &      { return data_; }           // 左辺値：参照で返す  
    const std::string& get() const& { return data_; }           // const 左辺値  
    std::string&&      get() &&      { return std::move(data_); } // 右辺値：ムーブで返す  
    const std::string&& get() const&& { return std::move(data_); } // const 右辺値  
};
```

- **左辺値**なら参照で返す（コピーさせない） / **右辺値**ならムーブして返す（奪ってよい）
- さらに **const の有無**で $\times 2 \rightarrow$ 合計 $2 \times 2 = 4$ つ

ほぼ同じコードの 4 連コピー。メンテもバグの温床。C++23 はこれを 1 つにできる。

deducing this — 自分自身を引数で受け取る C++23

C++23 では、メンバ関数の**第 1 引数として“自分自身”**を明示的に書ける。

```
struct Widget {  
    void print(this const Widget& self); // this を明示的に書く  
    // 呼び出しは今まで通り w.print(); (self は自動で渡る)  
};
```

- `self` が、今まで暗黙だった `this` (自分自身)
- 呼び出し側の書き方は**何も変わらない**

用語：deducing this (明示的オブジェクトパラメータ)

メンバ関数の先頭に `this ...` と書いて、**自分自身を引数として受け取る** C++23 の機能。

this auto&& self = オブジェクト版の転送参照

this の型を auto&& にすると、第 7 章の**転送参照をオブジェクト自身に使った形**になる。

```
struct S {  
    void f(this auto&& self); // self は呼び出し方に応じて型が変わる  
};
```

self は「どう呼ばれたか (左辺値/右辺値・const)」を**丸ごと覚える**。

呼び出し方	self の型
w.f() (左辺値)	Widget&
std::move(w).f() (右辺値)	Widget&&
const な w から	const Widget&

👉 これ 1 つで、さっきの **4 オーバーロード分の情報**を表せる。

std::forward_like — forward のメンバ版 C++23

第 8 章の `forward` は「**引数**」を値カテゴリ保って渡した。

`forward_like` は「**メンバ**」を、`self` (オブジェクト) の値カテゴリに合わせて返す。

```
#include <utility>
class Widget {
    std::string data_;
public:
    auto&& get(this auto&& self) { // これ 1 つで従来の 4 つを代替
        return std::forward_like<decltype(self)>(self.data_);
    }
};
```

- `decltype(self)` … `self` の型 (=呼び出し元の値カテゴリの「記憶」)
- 左辺値で呼ぶ → `data_` は**左辺値参照**で返る (コピーさせない)
- `std::move(w).get()` → `data_` は**右辺値参照**で返る (ムーブ可能) 

まとめ：C++23 は「forward の延長」

新機能に見えて、その正体は第 7・8 章の応用。

新機能	一言でいうと	元ネタ
deducing this	オブジェクト自身の転送参照	第 7 章 auto&&
std::forward_like	メンバ用の forward	第 8 章 std::forward

- まず目標は、モダンな標準ライブラリのコードで見かけたときに読めること
- 自分で書くな：getter の重複をなくす、CRTP を簡潔にする、等で効いてくる

10. スマートポインタと関数ラッパ

std::unique_ptr はムーブでしか渡せない C++11

唯一の所有者。コピー禁止 (コピーすると所有者が 2 人になり二重解放するため)。

```
auto p = std::make_unique<Mesh>(); // C++14: make_unique 推奨
// auto q = p;                      // NG: コピー不可 (deleteされている)
auto q = std::move(p);              // OK: 所有権を q へ移動 (p は nullptr に)
```

```
void consume(std::unique_ptr<Mesh> m); // 所有権を渡す = ムーブ前提
consume(std::move(q));                // 呼ぶ側も move を明示
```

```
void peek(const std::unique_ptr<Mesh>& m); // 所有権を渡さず「借りるだけ」なら参照
// もっと良いのは中身だけ借りる: void peek(const Mesh& m);
```

👉 所有権を移すなら `unique_ptr` を値で渡してムーブ。ただ使うだけなら `const&` (もしくは中身の参照)。

そもそも `std::function` とは C++11

用語： `std::function` = 関数・ラムダ・関数オブジェクトなど

「**呼び出せるもの (callable)**」を、型を問わず同じ変数に入れておける“関数の入れ物”。

```
#include <functional>

std::function<int(int)> f;          // 「int を受け取り int を返す」入れ物
f = [](int x){ return x + 1; };    // ラムダを入れる
int a = f(10);                    // 中身呼び出す → 11
f = [](int x){ return x * x; };    // 別の処理に差し替えも自由 → f(10) は 100
```

- コールバック / イベントハンドラ / タスクを **1つの型で持ち回れる**
- ゲームでは「あとで実行する処理」を配列に貯めておく、といった用途で頻出

問題：ムーブオンリーなラムダが入らない

ラムダが `unique_ptr` など **キャプチャ** すると、そのラムダは「ムーブは出来るがコピーは出来ない (= ムーブオンリー)」になる。

```
auto task = [p = std::make_unique<Mesh>()]() { /* p を使う */ };  
//           ↳ unique_ptr を抱えたラムダ → コピー不可  
  
std::function<void()> f = std::move(task); // X コンパイルエラー！
```

- `std::function` は **コピーできる中身しか入れられない**
- `unique_ptr` を持つラムダはコピー不可なので、弾かれてしまう

なぜ `std::function` はコピー可能を要求する？

`std::function` 自身が **コピーできる型** だから。

```
std::function<void()> a = someLambda;  
std::function<void()> b = a; // ← 入れ物ごとコピーできる
```

- 入れ物をコピーするなら、**中に入れた callable もコピーする必要がある**
- なので「入れる時点で copyable (コピー可能) を要求」する設計になっている

💡 「入れ物がコピーできる」なら「中身もコピーできないと辻褄が合わない」。
この制約が、ムーブオンリーな資源を持つラムダを閉め出していた。

解決: `std::move_only_function` C++23

用語: `std::move_only_function` = `std::function` の **ムーブ専用版**。
コピーは出来ない代わりに、**ムーブオンリーな callable も格納できる** (C++23)。

```
#include <functional>
auto task = [p = std::make_unique<Mesh>()]() { /* p を使う */ };

// std::function<void()> f = std::move(task);           // X
std::move_only_function<void()> f = std::move(task);    //  OK
f();
```

- コピーコンストラクタ / 代入は `= delete` (= **ムーブのみ**)
- `const` / `&&` / `noexcept` などの修飾もサポート
- `std::packaged_task` など「ムーブでしか運べないもの」も持てる

function との違い・使いどころ

	std::function	std::move_only_function
コピー	可能	不可 (= delete)
格納できる callable	copyable のみ	move-only もOK
導入	C++11	C++23
空のまま呼ぶと	例外 bad_function_call	未定義動作 (UB) ⚠

- ゲームでの出番：**タスクキュー / 遅延実行 / コールバック**で、`unique_ptr` などの資源をラムダに持たせたいとき
 - 従来は `shared_ptr` で無理やり copyable にして回避していたが、**所有権が明確で軽い** `move_only_function` の方が素直
- 👉 C++26 では逆に、コピー可能版を一般化した `std::copyable_function` も予定。

11. `emplace_back` とコンテナ

push_back vs emplace_back C++11

```
std::vector<Enemy> v;  
  
v.push_back(Enemy(100, "orc")); // ①一時オブジェクト生成 → ②それをムーブ格納  
v.emplace_back(100, "orc");     // コンストラクタ引数を渡し、要素をその場で直接構築
```

- `emplace_back` は**引数を完全転送**して、コンテナの領域内で**直接コンストラクト**
- 一時オブジェクトの生成 + ムーブ（またはコピー）を丸ごと省ける
- 内部でやっているのは §8 の完全転送そのもの

C++17 以降 `emplace_back` は**構築した要素への参照を返す**（それ以前は `void`）。

実務での注意：reserve を忘れない

`emplace_back` でも、容量が足りなければ**再確保 + 全要素の移動**が起きる。

```
std::vector<Enemy> v;  
v.reserve(1000);           // ★事前に容量確保 → 再確保ゼロ  
for (int i = 0; i < 1000; ++i)  
    v.emplace_back(/* ... */); // 途中の move/copy 連鎖が起きない
```

- ムーブが `noexcept` (§ 6) でないと、再確保時に**コピー**が走る点も併せて意識
- ゲームのように大量生成する場面では `reserve` + `noexcept` ムーブが効く

12. コピー省略

RVO / NRVO / 保証されたコピー省略

RVO / NRVO とは

関数が返す値を、**受け取る変数へ直接構築**してコピー/ムーブを消す最適化。

```
Buffer makeBuffer() {  
    return Buffer(1024); // RVO : 戻り値のローカル変数を、呼び出し側の変数に直接構築  
}  
Buffer makeNamed() {  
    Buffer b(1024);  
    return b;           // NRVO : 名前付きローカル b を直接構築 (コピー/ムーブ省略)  
}  
auto x = makeBuffer(); // コピーもムーブも走らない
```

- **RVO** … Return Value Optimization (無名の一時を返す)
- **NRVO** … Named RVO (名前付きローカルを返す)

保証されたコピー省略 C++17

C++17 で、prvalue（純粋右辺値）を返す/初期化する場合のコピー省略が「必須」になった。

```
Buffer makeBuffer() { return Buffer(1024); } // prvalue を返す
Buffer x = makeBuffer();                  // コピー/ムーブ「絶対に」起きない
```

- prvalue は「一時オブジェクトを作ってからコピー」ではなく、必要になる瞬間まで**実体化されない (materialization)** 概念に変わった
- そのため、**コピー/ムーブコンストラクタが `= delete` でも返せる**
- 一方、**NRVO（名前付き）は C++23 でも「任意（保証なし）」** … コンパイラ次第

アンチパターン： `return std::move(local);`

「ムーブした方が速い」 と思って付けると、 **むしろ遅くなる**。

```
Buffer bad() {  
    Buffer b(1024);  
    return std::move(b); // ✗ NRVO を阻害。xvalue になり最適化が外れてムーブ強制  
}  
Buffer good() {  
    Buffer b(1024);  
    return b;           // ✓ そのまま返す (NRVO が効けばコピー/ムーブすら消える)  
}
```

ローカル変数をそのまま返すときは **`std::move` を付けない**。

例外：メンバ変数や引数を返す等、NRVO が元々効かない場面のみ `std::move` が有効。

13. まとめ

move / forward チートシート

場面	使うもの
これ以上使わないローカル値・具体的な T&& を右辺値化	std::move
転送参照 T&& (型推論あり) の値カテゴリを保って転送	std::forward<T>
メンバを self の値カテゴリに合わせて転送 (C++23)	std::forward_like<decltype(self)>
コピーしたくない汎用の受け (範囲 for 等)	auto&&
変更せず借りるだけ	const T&
所有権を移す	値渡し + std::move (unique_ptr 等)

落とし穴チェックリスト

- ⚠ **ムーブ** ≠ `std::move` (`std::move` はキャスト = 許可証)
- ⚠ **use-after-move** : ムーブ後の中身を読まない (valid but unspecified)
- ⚠ `const` は**ムーブできない** : 安易な `const` がムーブを殺す
- ⚠ **ムーブ**は `noexcept` : 付けないと `vector` 再確保でコピーに逆戻り
- ⚠ `return std::move(local)` は**書かない** : NRVO を阻害する
- ⚠ 転送参照には `forward`、ローカル値には `move` を混同しない
- ⚠ `unique_ptr` は**コピー不可** : 所有権はムーブでのみ移動

全体像 (バージョン別)

機能	導入
右辺値参照 <code>T&&</code> / <code>std::move</code> / <code>std::forward</code> / ムーブ特殊メンバ / <code>noexcept</code> / <code>unique_ptr</code>	C++11
<code>std::make_unique</code> / <code>std::exchange</code>	C++14
保証されたコピー省略 / <code>emplace_back</code> が参照を返す	C++17
条件付きトリビアル特殊メンバ	C++20
<code>deducing this</code> / <code>std::forward_like</code> / <code>std::move_only_function</code>	C++23

おまけ：C++26 では `std::copyable_function` / `std::function_ref` など関数ラップの拡張が進行中。

参考リンク

- ムーブセマンティクス総論：cpprefjp `rvalue_ref_and_move_semantics`
- `std::forward` リファレンス：cpprefjp `utility/forward`
- `std::forward_like` : cpprefjp `utility/forward_like`
- `std::move_only_function` : cpprefjp `functional/move_only_function`
- 保証されたコピー省略：cpprefjp `cpp17/guaranteed_copy_elision`
- ムーブと完全転送の解説：proc-cpuinfo / komorinfo / zenn(mafafa) / qiita(alphya)

おつかれさまでした 🎮